

## -----MONGODB RAW QUERIES -----

document format

```
{ "first name": "Daniyal",  
  "address": { "city": "C", "house": "H" }}
```

```
db.users.insertOne({ name: 'Ali', age: 22 })  
db.users.insertMany([{ name: 'A', age: 20 },  
{ name: 'B', age: 25 }])
```

```
db.users.find()  
db.users.find({ age: 22 })  
db.users.find({ age: { $gt: 20 } })  
db.users.find({ city: 'Islamabad', age: 22 })  
db.users.find(  
  { $or: [{ age: 20 }, { city: 'Lahore' }] },  
  { name: 1, age: 1, _id: 0  
})  
db.users.find().sort({ age: 1 }) // asc  
db.users.find().sort({ age: -1 }) // desc  
db.users.find().limit(5)
```

```
db.users.updateOne({ name: 'Ali' }, { $set:  
{ age: 23 } })  
db.users.updateMany({ age: 20 }, { $set:  
{ active: true } })
```

\$set: set field value  
\$unset: remove field  
\$inc: increment number  
\$push: add to array  
\$pull: remove from array  
\$currentDate:

```
db.users.deleteOne({ name: 'Ali' })  
db.users.deleteMany({ age: { $lt: 18 },  
{ "trial": { $exists: true, $eq: true } }  
{ role: { $nin: ["admin", "moderator"] } },  
})
```

// Operators: \$gt \$lt \$gte \$lte \$eq/ne  
\$in/nin \$and/or/nor/not \$exists

### Aggregation / JOIN (\$lookup)

```
db.articles.aggregate([  
  { $lookup: { from: 'User',  
    localField: 'Author',  
    foreignField: '_id', as: 'user' } },  
  { $unwind: '$user' },  
  { $project: { title: 1,  
    Author: '$user.Name' } }  
])
```

### Indexing

```
db.users.createIndex({ lastName: 1 })  
db.users.createIndex({ email: 1 },  
{ unique: true, name: 'EmailIdx' })
```

## — MONGODB CONNECTION + MODEL —

```
const mongoose = require('mongoose');  
mongoose.connect('mongodb://127.0.0.1:27017/  
myDB')  
  .then(() => console.log('Connected'))  
  .catch(err => console.log(err));  
const userSchema = new mongoose.Schema({  
  name: String, age: Number, email: String  
});  
const User = mongoose.model('User',  
userSchema);
```

## — EXPRESS ROUTES —

### Request Object

```
req.params: path params (/user/:id)  
req.query: query string (?name=Ali)  
req.body: request body (POST/PUT)  
req.get(h): get header value
```

### Response Methods

```
res.send(): send HTML/text  
res.json(obj): send JSON (sets content-type)  
res.status(n): set HTTP status code  
res.type('text'): set content-type  
res.redirect(url): redirect  
res.sendStatus(n): status + default tex
```

### Middle ware:

```
1. Application-level (runs on all routes)  
app.use((req, res, next) => {  
  next();  
});  
2. Route-specific (specific routes)  
app.use('/api', (req, res, next) => {  
  next();  
});  
3. Error-handling (catches errors)  
app.use((err, req, res, next) => {  
  res.status(500).send("Something broke!");  
});
```

```
app.get('/api/users', async (req, res) => {  
  try {  
    const users = await User.find();  
    res.json(users);  
  } catch (e) {}  
  { res.status(500).json({ error: e.message }) }  
});  
  
app.post('/api/users', async (req, res) => {  
  try {  
    const user = new User(req.body);  
    await user.save();  
    res.json(user);  
  } catch (e) {}  
  { res.status(500).json({ error: e.message }) }  
});  
  
app.put('/api/users/:id', async (req, res) => {  
  try {  
    const user = await  
    User.findByIdAndUpdate(  
      req.params.id, req.body, { new: true });  
    res.json(user);  
  } catch (e) {}  
  { res.status(500).json({ error: e.message }) }  
});  
  
app.delete('/api/users/:id', async (req, res)  
=> {  
  try {  
    await  
    User.findByIdAndDelete(req.params.id);  
    res.json({ message: 'User deleted' });  
  } catch (e) {}  
  { res.status(500).json({ error: e.message }) }  
});
```

## Express Router (Modular)

```
// users.js  
const router = express.Router()  
router.get('/', (req, res) => { ... })  
router.post('/', (req, res) => { ... })  
module.exports = router  
// app.js  
const users = require('./users')  
app.use('/users', users)  
  
// JWT Auth  
const jwt = require('jsonwebtoken')  
const token = jwt.sign({ user }, secret)  
const payload = jwt.verify(token, secret)  
// Auth header: Bearer <token>
```

## — REACT FULL APP (CLIENT SIDE) —

```
const App = () => {  
  const [users, setUsers] = useState([]);  
  const [name, setName] = useState('');  
  const [age, setAge] = useState('');  
  
  useEffect(() => { fetchUsers(); }, []);  
  
  const fetchUsers = async () => {  
    const res = await fetch('/api/users');  
    const data = await res.json();  
    setUsers(data);  
  };  
  
  const addUser = async () => {  
    const res = await fetch('/api/users', {  
      method: 'POST',  
      headers:  
{ 'Content-Type': 'application/json' },  
      body: JSON.stringify({ name,  
age: Number(age) })  
    });  
    const newUser = await res.json();  
    setUsers([...users, newUser]);  
  };  
  
  const updateUser = async (id) => {  
    await fetch(`/api/users/${id}`, {  
      method: 'PUT',  
      headers:  
{ 'Content-Type': 'application/json' },  
      body: JSON.stringify({ name,  
age: Number(age) })  
    });  
    fetchUsers();  
  };  
  
  const deleteUser = async (id) => {  
    await fetch(`/api/users/${id}`,  
{ method: 'DELETE' });  
    setUsers(users.filter(u => u._id !==  
id));  
  };  
};
```

```
};  
  
return (  
  <div>  
    <input value={name}  
      onChange={e => setName(e.target.value)}  
      placeholder='Name' />  
    <input value={age}  
      onChange={e => setAge(e.target.value)}  
      placeholder='Age' />  
    <button onClick={addUser}>Add</button>  
  
    { users.map(user => (  
      <div key={user._id}>  
        { user.name } - { user.age }  
        <button  
onClick={() => updateUser(user._id)}>  
Update</button>  
        <button  
onClick={() => deleteUser(user._id)}>  
Delete</button>  
      </div>  
    )) }  
  </div>  
);  
};
```

## — REACT EXTRAS —

```
{ loading && <p>Loading...</p> }  
{ isLoggedIn ? <Dashboard> : <Login> }  
// Props  
<UserCard name='Ali' age={22} />  
const  
UserCard = ({ name, age }) => <p>{ name } - { age }</p>  
<input onChange={e => setVal(e.target.value)} />  
Lists & Keys  
const items = ['Apple', 'Banana', 'Cherry']  
<ul>  
  { items.map((item, i) =>  
    <li key={i}>{ item }</li> ) }  
</ul>
```

// Keys must be unique siblings

### Controlled Forms

```
const [name, setName] = useState('')  
<input value={name}  
  onChange={e => setName(e.target.value)} />  
<button onClick={handleSubmit}>Send</button>  
Uncontrolled Forms (ref)  
const inputRef = React.createRef()  
<input type='text' ref={inputRef} />
```

```
fetch('http://api.example.com/users')  
  .then(res => res.json())  
  .then(data => setState({ data }))  
  .catch(console.log)
```

## — REACT HOOKS —

```
const [count, setCount] = useState(0);
```

```
// useEffect  
useEffect(() => { fetchUsers(); }, []);  
// ^ on mount only  
useEffect(() => { fetch(); }, [id]);  
// ^ when id changes  
useEffect(() => { fetch(); });  
// ^ every render
```

```
// useEffect - BEFORE browser paints  
// use for DOM measurements  
useLayoutEffect(() => {  
  const h = divRef.current.offsetHeight;  
}, []);
```

```
// useRef - reference DOM element  
const inputRef = useRef(null);  
inputRef.current.focus();  
<input ref={inputRef} />
```

```
// useContext - share data without props  
const ThemeContext = createContext();  
// Parent:  
<ThemeContext.Provider value='dark'>  
  <Child />  
</ThemeContext.Provider>  
// Child:  
const theme = useContext(ThemeContext);
```

```
// useReducer - complex state  
const reducer = (state, action) => {  
  switch (action.type) {  
    case 'inc': return { count: state.count + 1 };  
    case 'dec': return { count: state.count - 1 };  
    default: return state;  
  }  
};  
const [state, dispatch] = useReducer(reducer,  
{ count: 0 });  
dispatch({ type: 'inc' });
```

```
// useMemo - cache a VALUE
const total = useMemo(() => {
  return nums.reduce((a,n) => a+n, 0);
}, [nums]);

// useCallback - cache a FUNCTION
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
// useMemo vs useCallback:
// useMemo → VALUE → useMemo(()=>calc(), [d])
// useCallback → FUNC →
useCallback(()=>fn(), [d])
```

```
// useEffect vs useLayoutEffect:
// useEffect → AFTER paint → async →
most cases
// useLayoutEffect → BEFORE paint → sync →
DOM
```

#### Class Component:

```
class Counter extends React.Component {
  constructor(props) {
    super(props)
    this.state = {count:0}
  }
  render() {
    return <button onClick={()=>
      this.setState({count:this.state.count+1})}
    >{this.state.count}</button>
  }
}
```

#### Lifecycle (Class)

componentDidMount: after first render (API calls)  
 componentDidUpdate: after re-render  
 componentWillUnmount: cleanup before removal

```
componentDidMount() {
  fetch('/api/data')
    .then(r=>r.json())
    .then(d=>this.setState({data:d}))
}
```

```
<Router>
<Link to='/about'>About</Link>
<Routes>
<Route path="/" element={<Home/>}/>
<Route path='/about' element={<About/>}/>
<Route path='*' element={<NotFound/>}/>
</Routes>
</Router>
```

```
const navigate = useNavigate()
navigate('/dashboard')
```

```
<Route path='/user/:id' element={<User/>}/>
import {useParams} from 'react-router-dom'
const {id} = useParams()
```

#### AXIOS:

```
axios.get('/api/users').then(res=>{
  setState({data:res.data})
})
axios.post('/api',{name,msg})
.then(()=>navigate('/'))
```

#### Context-API

```
const ThemeCtx = React.createContext('light')
// Provider (parent):
<ThemeCtx.Provider value='dark'>
<App/>
</ThemeCtx.Provider>
// Consumer (child):
const theme = useContext(ThemeCtx)
```

#### ----- ANGULAR -----

```
let arr: number[] = [1,2,3]
function add(a:number, b:number): number {
  return a+b }
```

#### @COMPONENT

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'myApp';
}
```

#### — @NGMODULE —

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule, RouterModule],
  bootstrap: [AppComponent],
  providers: [MyService]
})
export class AppModule {}
```

#### @Injectable Service

```
@Injectable({ providedIn: 'root' })
export class DataService {
  constructor(private http: HttpClient){}
  getData(): Observable<Item[]> {
    return this.http.get<Item[]>('/api/items')
  }
  postData(item:Item): Observable<Item> {
    return this.http.post<Item>('/api',item)
  }
}
// Navigate in Code// Inject in component
constructor(private ds: DataService){}
ngOnInit(){
  this.ds.getData().subscribe(
    data => this.items = data)
}
```

#### @Input() — Parent → Child

```
// child.component.ts
import {Input} from '@angular/core'
@Input() title: string = ''
@Input() items: Item[] = []
```

```
// parent template
<app-child [title]='myTitle'
[items]='myItems'>
</app-child>
```

#### @Output() — Child → Parent

```
// child.component.ts
import {Output,EventEmitter} from '@angular/core'
@Output() itemAdded =
new EventEmitter<Item>()
addItem() {
  this.itemAdded.emit(newItem)
}
```

```
// parent template
<app-child (itemAdded)='onAdd($event)'>
</app-child>
// parent component
onAdd(item: Item) {
  this.items.push(item)
}

— *ngFor & *ngIf —
<div *ngFor="let user of users">
  {{ user.name }}
</div>
```

```
<p *ngIf="isLoggedIn">Welcome!</p>
<p *ngIf="x; else noX">Has X</p>
```

```
<li *ngFor="let i of items;
let idx=index">{{idx}}.{{i}}</li>
<!-- ngSwitch -->
<div [ngSwitch]='day'>
  <span *ngSwitchCase='1'>Mon</span>
  <span *ngSwitchDefault>Other</span>
</div>
```

#### — BUILT-IN PIPES —

```
{{ name | uppercase }}
{{ name | lowercase }}
{{ date | date:'shortDate' }}
{{ price | currency:'USD' }}
{{ name | slice:0:3 }}
```

#### — CUSTOM PIPE —

```
@Pipe({ name: 'exclaim' })
export class ExclaimPipe implements PipeTransform {
  transform(value: string): string {
    return value + '!!!';
  }
}
// Usage: {{ 'hello' | exclaim }} → hello!
```

#### — ROUTING —

```
RouterModule.forRoot([
  { path: 'home', component: HomeComponent },
  {
    path: '',
    redirectTo: 'home',
    pathMatch: 'full'
  }
])
```

```
// In app template:
<router-outlet />
```

```
// Navigation link:
<a routerLink="/home">Home</a>
```

#### — DECORATOR SUMMARY —

@Component → defines a component  
 @NgModule → organizes app modules  
 @Injectable → marks class as service  
 @Input() → receive data from parent  
 @Output() → emit events to parent  
 @Pipe → transform template data

#### — QUICK REFERENCE —

[] → property binding (parent→child)  
 () → event binding (child→parent)  
 [()] → two-way binding (ngModel)  
 {{{}} → interpolation

\* → structural directive  
 | → pipe operator

#### Attribute directive

```
<p [ngStyle]="{color:'red'}">Text</p>
<p [style.color]=" 'blue' ">Text</p>
<p [ngClass]="{active:isActive,
disabled:!isActive}">Text</p>
```

#### Template Syntax

```
{{ expr }}: Interpolation
[prop]='expr': Property binding
(event)='fn()': Event binding
[(ngModel)]='prop': Two-way binding
#varName: Template reference variable
```

```
<input [(ngModel)]='name'>
<p>Hello {{name}}</p>
<button (click)='save()'>Save</button>
<img [src]='imageUrl'>
```

#### HttpClient (import HttpClientModule)

```
// GET this.http.get('/api/items')
// POST this.http.post('/api',body)
// PUT this.http.put('/api/id',body)
// DELETE this.http.delete('/api/id')
```

#### Angular Routing

```
// app-routing.module.ts
const routes: Routes = [
  {path:'',component:HomeComponent},
  {path:'about',component:AboutComponent},
  {path:'user/:id',component:UserComponent},
  {path:'**',component:NotFoundComponent}
]
```

```
// template navigation
<a routerLink='/about'>About</a>
<a [routerLink]="['/user',userId]">

// read route params
constructor(private route: ActivatedRoute){}
ngOnInit(){
  const id = this.route.snapshot
  .paramMap.get('id')
}
// navigate in code
constructor(private router: Router){}
this.router.navigate(['/home'])
```

#### router-outlet & routerLink

```
// app.component.html
<nav>
  <a routerLink="/">Home</a>
  <a routerLink="/about">About</a>
</nav>
<router-outlet></router-outlet>
```

#### Dependency Injection

providers[]: in @NgModule or @Component  
 providedIn:'root': singleton app-wide  
 constructor(private s:Service){}: inject  
 // Provider with value token  
 providers:[{provide:'API\_URL',
 useValue:'http://localhost:3000'}]  
 // Inject with @Inject  
 constructor(@Inject('API\_URL') url:string)

#### Key Decorators Summary

@Component: defines a component (selector,template,style)  
 @NgModule: defines a module (declarations,imports,exports)  
 @Injectable: marks class for DI  
 @Input(): receive data from parent  
 @Output(): emit events to parent  
 @Inject(): inject non-class tokens

#### Lifecycle Hooks

ngOnInit: after component init (use for API calls)  
 ngOnChanges: when @Input values change  
 ngOnDestroy: cleanup before removal  
 ngAfterViewInit: after view initialized

#### Dev Proxy Setup

```
// proxy.conf.json
{"api":{"target":
"http://localhost:3000",
"secure":false}}
// package.json scripts:
"start":"ng serve
--proxy-config proxy.conf.json"
```

#### Environment Variables

```
// .env file
DBURL=mongodb://localhost/mydb
JWT_SECRET=mysecret
// In Node.js
process.env.DBURL
require('dotenv').config()
```

